touying-ufac-unofficial

Lecture slides for the Federal University of Acre (UFAC).

v0.3.0 November 03, 2026

https://github.com/lucaslrodri/touying-ufac-unofficial

Lucas Lima Rodrigues

A Touying theme for lecture slides, following the visual identity of the Federal University of Acre. Slides are written with headings and a few Markdown-like shorthands for boxes, quotations, arrow lists and emphases. This manual is set with the rules of the theme itself.

Contents

1 Quick start 2
1.1 The information of the deck 2
1.2 The theme and the cover 4
1.3 Logos 9
1.4 Language 10
1.5 Dependencies 11
2 Slides and headings 12
2.1 Sections 12
2.2 Outline 13
2.3 Slides and continuations 13
2.4 Exercises and examples 14
2.5 Subtitles 17
2.6 Slide functions 19
3 Inline formatting and shorthands 23
3.1 Bold, italic and alert 23
3.2 Highlight and underline 23
3.3 Code chips 23
3.4 Colors by context 24
3.5 Colored emphases 24
3.6 Escapes 25
4 Lists and terms 26
4.1 Arrow lists 26
4.2 Bullet lists 26
4.3 Numbered lists 26
4.4 Terms 27
5 Boxes and quotes 28
5.1 Emphasis boxes 28
5.2 The box shorthand 29
5.3 Equation boxes 29
5.4 Callouts 30
5.5 Tags 31
5.6 Quotations 32
6 Columns 34
7 Colors and logo 35
8 Tables, figures, math, code and icons 37
8.1 Tables 37
8.2 Figures and math 37
8.3 Code 37
8.4 Icons 39
9 Touying features 41
10 Index of definitions 43

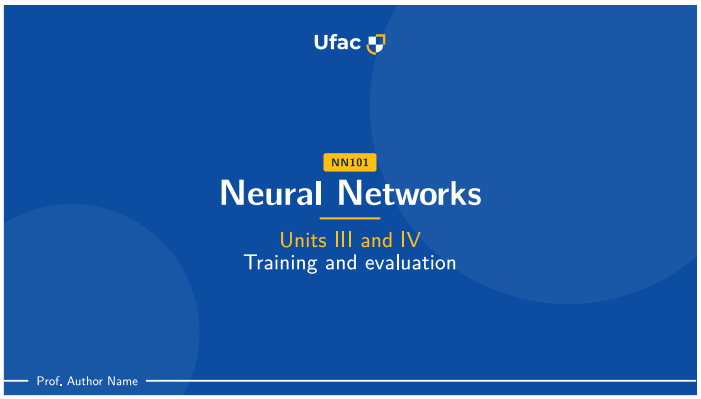
1 Quick start

A deck imports Touying and the theme, applies `ufac-theme` and usually starts with the cover:

```
#import "@preview/touying:0.7.4": *
#import "@preview/touying-ufac-unofficial:0.3.0": *
```

```
#show: ufac-theme.with(
  aspect-ratio: "16-9",
  lang: "en",
  config-info(
    title: [Neural Networks],
    short-title: [NN101],
    subtitle: [Training and
evaluation],
    short-subtitle: [Units III and
IV],
    author: [Prof. Author Name],
    counter-prefix: [1.],
  ),
)

#title-slide()
```



`typst init @preview/touying-ufac-unofficial:0.3.0` creates a deck from the package template, and `example/main.typ` shows the elements of the syntax in a short deck. From here on, every example shows the source on the left and the result on the right: a real slide when the feature belongs to the slide (headers, pills, continuations), otherwise the content alone, laid out by the same rules as in a deck.

1.1 The information of the deck

`config-info` is Touying's; the theme reads these fields:

Field	Default	What it is and where it appears
`title`	`[]`	the name of the course or of the work: large on the cover, and in the footer
`short-title`	`none`	its code or a shorter name: before the title in the footer ("CODE ▪ TITLE")
`subtitle`	`[]`	the topic of the lecture: on the cover
`short-subtitle`	`none`	the lecture or the unit ("Lecture 1", "Unit II"): on the cover, in yellow
`author`	`none`	one, an array or free content (below): on the cover and on the right of the footer
`date`	`none`	`auto` (today), a `datetime` (written in full in the language of the deck) or a text
`institution`	`auto`	one or an array; `auto` is the Universidade Federal do Acre; only for `#info`
`logo`	`auto`	one or an array, in the footer and on the cover (see Logos below)
`contact`	`none`	one or an array (an e-mail, a page)
`counter-prefix`	`[1.]`	prefix of the exercise and example numbers: `[1.]` gives "Exercise 1.N", `none` gives "Exercise N"

In a course, some fields have clearer names, which are aliases of the fields above (two names of one field are an error). A date given as `semester` or `year` is short, and also goes to the footer, after the title.

Alias	Field
`subject`	`title`
`subject-code`	`short-title`
`unit`	`short-subtitle`
`unit-title`	`subtitle`
`course` (the program, in a final paper)	`short-subtitle`
`semester`, `year`	`date`

`kind` tells what the presentation is, and brings the recommended style of the cover, the label of the authors and the tag of the cover; anything the deck writes wins over it.

`kind`	Recommends
`"lecture"`, `"talk"`	the style `"line"`
`"course"`	the style `"top"` and the label "Ministrantes" / "Instructors" over the authors
`"coursework"`	the style `"stack"`
`"tcc"`, `"dissertation"`, `"thesis"`	the style `"top"` and the tag of the work ("Trabalho de Conclusão de Curso", …)

An author can be written as free content: the lines broken with `\`, and a line only in emphasis is a label. The cover shows it as written; the footer and the metadata of the PDF get the names of the first group, those who present.

```
config-info(author: [_Students_ \ First Author \ Second Author \ _Advisor_ \ Prof. Dr. Advisor Name])
```

`info()`

```
info(
  key str function
  height: auto auto length
  white: false bool
) -> content
```

An information of the deck, ready to show, anywhere in a slide (inside boxes and tables too): `#info("author")` gives "A, B e C", `#info("date")` the date in full, `#info("logo", height: 2em)` the logos. The aliases are keys too: `#info("subject")` is the `title`. With a function, it receives the dictionary of all the information: `#info(i => [#i.short-title: #i.title])`.

```
The course #info("title"), taught by #info("author").
```

Argument

key — `str` | `function`

`"title"`, `"short-title"`, `"subtitle"`, `"short-subtitle"`, `"author"`, `"date"`, `"institution"`, `"logo"`, `"contact"` or an alias (`"subject"`, `"semester"`, …); or a function of the dictionary.

Argument

height: auto — `auto` | `length`

Only for `"logo"`: the height of the logos; `auto` is the height in the footer (0.64em).

Argument

`white: false` `bool`

Only for `"logo"`: white logos, as on the cover.

1.2 The theme and the cover

`ufac-theme()`

```
ufac-theme(
  aspect-ratio: "16-9" str
  lang: "pt-br" str
  exercise-name: auto auto content
  example-name: auto auto content
  exercise-icon: "pencil" none str content symbol
  example-icon: "beaker" none str content symbol
  part-name: auto auto content
  footer-left: auto auto content function
  footer-right: auto auto content function
  section-margin: true bool length
  background: "circles" str
  codly: none none module
  code-languages: (:) dictionary
  ..args dictionary
  body content
) -> content
```

The theme. It sets the page (22pt New Computer Modern Sans, the UFAC header and footer) and every rule of the syntax, and takes Touying's configs as extra arguments.

```
#show: ufac-theme.with(
  aspect-ratio: "16-9",
  config-info(
    title: [Neural Networks], short-title: [NN101],
    subtitle: [Training and evaluation], short-subtitle: [Units III and IV],
    author: [Prof. Name], counter-prefix: [1.],
  ),
)
```

`config-info` carries the fields of Touying: `title` (the name of the course or of the work), `short-title` (its code or a shorter name: the tag of the cover), `subtitle` (the topic), `short-subtitle` (the lecture or the unit), `author` (one, an array or free content), `date`, `institution`, `logo` and `contact`; the aliases of a course (`subject`, `subject-code`, `unit`, `unit-title`, `course`, `semester`, `year`); the `kind` of presentation; and `counter-prefix`, the prefix of the exercise and example numbers ("Exercise 1.N"; `none` gives "Exercise N").

Argument

`aspect-ratio: "16-9"` `str`

Any ratio accepted by Touying's `utils.page-args-from-aspect-ratio`: `"16-9"`, `"4-3"`, …

Argument

`lang: "pt-br"` `str`

Language of the deck: `"pt-br"` (or `"pt"`), `"en"` or `"es"`, optionally with a region (`"en-US"`). It sets `text.lang`/`region` and the localized names below.

Argument

`exercise-name: auto` — `auto` | `content`

Name in the pill of exercise slides; `auto` is Exercício / Exercise / Ejercicio.

Argument

`example-name: auto` — `auto` | `content`

Name in the pill of example slides; `auto` is Exemplo / Example / Ejemplo.

Argument

`exercise-icon: "pencil"` — `none` | `str` | `content` | `symbol`

Icon before the name in the pill of exercise slides: the name of an `icon`, any content (an image or a symbol, centered on the capitals like the icons) or `none` for no icon. `exercise-slide(icon:)` changes it on one slide.

Argument

`example-icon: "beaker"` — `none` | `str` | `content` | `symbol`

Icon before the name in the pill of example slides, like `exercise-icon`.

Argument

`part-name: auto` — `auto` | `content`

Prefix of the section slide ("Part N:"); `auto` is Parte / Part / Parte.

Argument

`footer-left: auto` — `auto` | `content` | `function`

Left text of the footer; a function receives `self`. `auto` is the `short-title` and the `title` separated by a dot and, when the date is a `semester` or a `year`, the date too ("AST101 ▪ Astronomy ▪ 2026.2").

Argument

`footer-right: auto` — `auto` | `content` | `function`

Right text of the footer; a function receives `self`. `auto` is the authors in running text ("A, B and C"); of an author written as free content, the names of its first group.

Argument

`section-margin: true` — `bool` | `length`

Distance between the content of the section slides (what follows `=`, on the right) and the right edge of the page. `true` keeps the page margin (`1.8em`, the same as the text); `false` puts it flush with the edge, which suits a photo or a figure with a background; a length sets the distance.

Argument

`background: "circles"` — `str`

Background of the cover and of the closing slide: `"circles"` (the solid UFAC blue with two translucent circles), `"gradient"` (a blue gradient, with the circles) or `"plain"` (the blue alone, the cover of 0.2.0).

Argument

`codly: none` — `none` | `module`

The codly 2 module, when the deck imports it (`#import "@preview/codly:2.0.0" as codly`): the code blocks then go through codly, with the look of the theme (`codly-ufac-theme`). `none` gives the code box of the theme, without codly.

Argument

`code-languages: (:)` — `dictionary`

Languages added to those of the theme (`code-languages`) or replacing them, as `(name:, icon:)`; the dictionary of the codly-languages package works too.

Argument

`..args` — `dictionary`

Touying configs, typically `config-info(..)`; also `config-common(..)`, `config-page(..)`.

Argument

`body` — `content`

The deck.

`title-slide()`

```
title-slide(
  style: auto  auto  str
  author-label: auto  auto  none  content
  header-left: auto  auto  none  content  function
  header-center: auto  auto  none  content  function
  header-right: auto  auto  none  content  function
  footer-left: auto  auto  none  content  function
  footer-center: auto  auto  none  content  function
  footer-right: auto  auto  none  content  function
  footer-line: auto  auto  bool
  ..body  content
) -> content
```

Cover: the blue page (`ufac-theme(background:)`) with the logos large at the top; in the centre, the tag (`short-title`), the `title`, the yellow rule, the `short-subtitle` in yellow, the `subtitle` and the content given to it, centered both ways in the room between the top and the bottom; the authors and the date where the `style` puts them. The institution does not appear (it is for `#info`). The slide counter is frozen, so the cover does not count.

Each position takes `auto` (what the style puts there), `none` (nothing), content (such as `#info("date")`) or a function of `self`, like `footer-left` of the theme. The text of the positions is 0.7 of the size of the slide.

```
#title-slide()
#title-slide(style: "top", footer-center: none, footer-right: info("date"))
#title-slide[#image("lab-white.svg", height: 1.7em)]
```

Argument

`style: auto` — `auto` | `str`

`"line"`: the authors on one line at the bottom left, the white line and the date on the right, with the logos in the centre of the top. `"stack"`: the authors on several lines at the bottom left and the date on the right, without the line. `"top"`: the authors at the top right and the date in the centre of the bottom, without the line. `auto` is the one of the `kind` of the deck, or `"line"`.

Argument

`author-label: auto` — `auto` | `none` | `content`

Label over the authors (“Instructors”); before them in `"line"`. `auto` is the one of the `kind`, or none.

Argument

`header-left: auto` — `auto` | `none` | `content` | `function`

What goes at the top left.

Argument

`header-center: auto` — `auto` | `none` | `content` | `function`

What goes at the top centre.

Argument

`header-right: auto` — `auto` | `none` | `content` | `function`

What goes at the top right.

Argument

`footer-left: auto` — `auto` | `none` | `content` | `function`

What goes at the bottom left.

Argument

`footer-center: auto` — `auto` | `none` | `content` | `function`

What goes at the bottom centre.

Argument

`footer-right: auto` — `auto` | `none` | `content` | `function`

What goes at the bottom right.

Argument

`footer-line: auto` — `auto` | `bool`

The white line at the bottom, like the one of the footer of the slides: from edge to edge, passing between what is at the bottom, at the height of the line of the footer. `auto` is the one of the style.

Argument

`..body` — `content`

Content in the centre, below the titles, `#title-slide[..]`; `#title-slide()` has none.

The cover has three bands: the logos at the top, the titles in the centre and the authors and the date at the bottom. The `style` decides where the authors and the date go: `"line"` (the default) puts them on one line at the bottom, with the white line of the footer passing between them and the logos in the centre of the top; `"stack"` puts the authors on several lines at the bottom left; `"top"` puts them at the

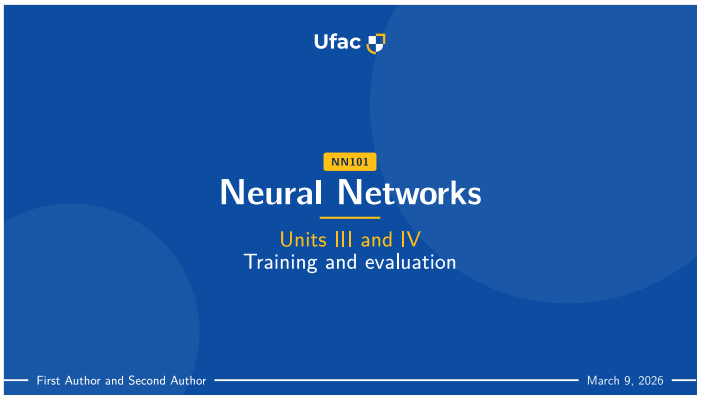
top right and the date in the centre of the bottom. Each of the six positions can be changed on its own, with content, `none`, a function of `self` or `#info`.

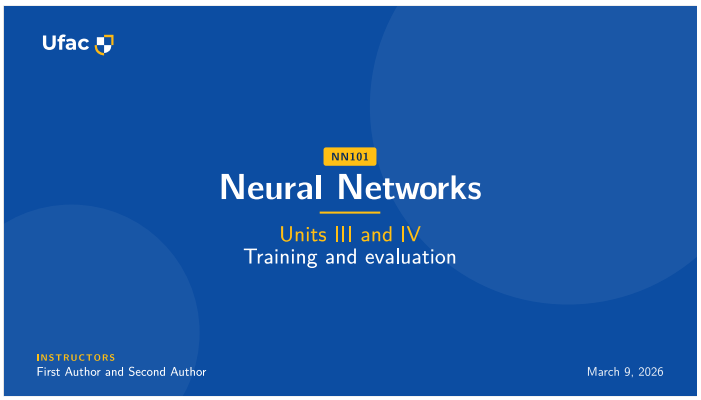
```
#title-slide()   // style: "line"

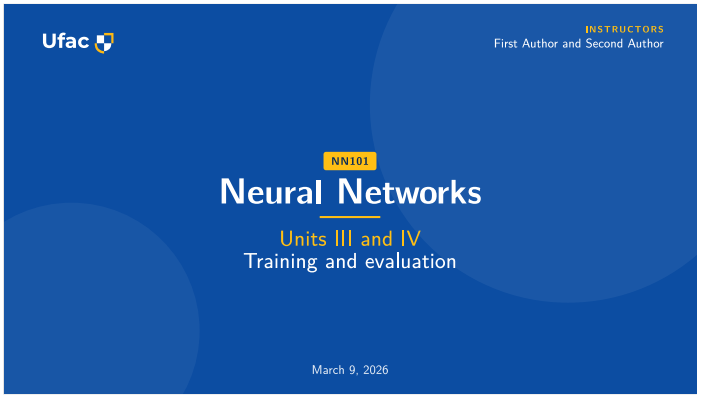
#title-slide(style: "stack",
author-label: [Instructors])

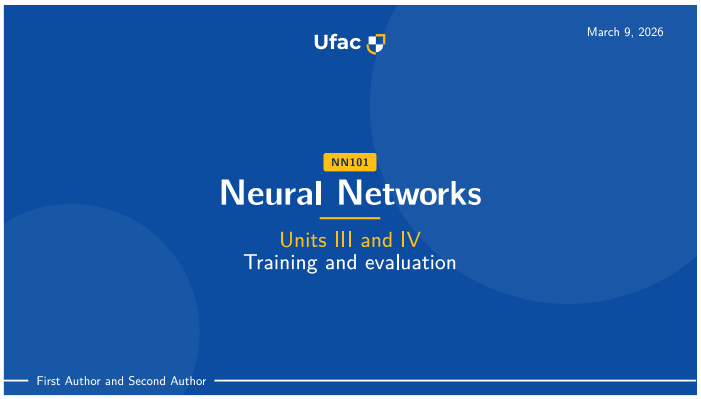
#title-slide(style: "top", author-
label: [Instructors])

#title-slide(footer-right: none,
header-right: info("date"))
```









The background of the cover and of the closing slide is a parameter of the theme: `"circles"` (the default), `"gradient"` or `"plain"`. A `kind` brings the recommended style, label and tag; an author written as free content keeps its lines and labels on the cover.

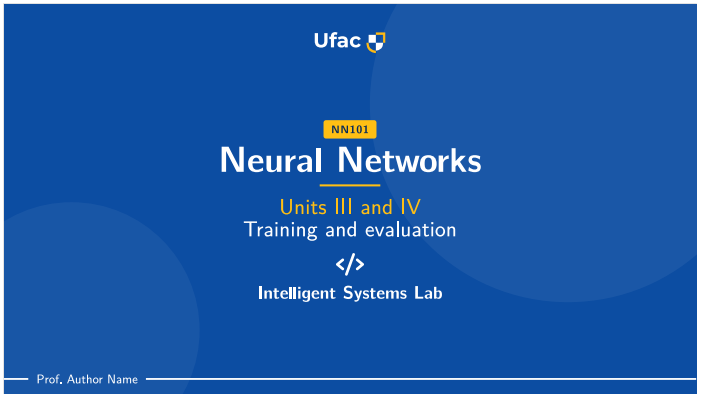
```
#show: ufac-theme.with(
  lang: "en",
  background: "gradient",   // "circles" (default), "gradient", "plain"
  config-info(
    kind: "tcc",
    title: [Recognising bird songs in city parks],
    course: [Biology],
    author: [_Student_ \ First Author \ _Advisor_ \ Prof. Dr. Advisor Name],
    year: 2026,
  ),
)

#title-slide()
```



Content given to the cover goes below the titles, in white, as in a pill: a partner's logo, the name of a lab or of a program.

```
#title-slide[
  #icon("code", size: 1.6em) \
  *Intelligent Systems Lab*
]
```



1.3 Logos

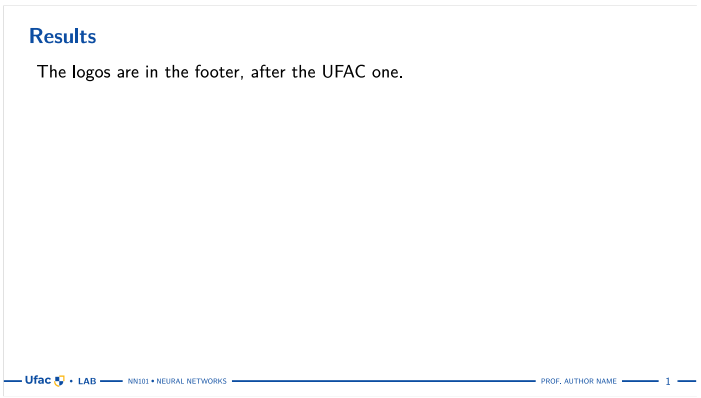
The footer and the cover show the UFAC logo. `config-info(logo:)` is the list of logos shown there, separated by dots: `(auto, x)` keeps the UFAC logo and adds `x` after it, `(x,)` replaces it and `none` removes them. A logo is content, or a function that receives the colour of the footer (blue on the slides, white on the cover), so that one drawing serves both; `auto` stands for the UFAC logo. A logo about `0.64em` high matches the UFAC one. Outside the footer, as in a handout that goes with the slides, `ufac-logo-image` draws the UFAC logo in the same colours (Section 7).

```typst
#show: ufac-theme.with(
  lang: "en",
  config-info(
    logo: (auto, color =>
text(size: 0.5em, weight: "bold",
fill: color)[LAB]),
    title: [Neural Networks],
    short-title: [NN101],
    subtitle: [Training and
evaluation],
    author: [Prof. Author Name],
  ),
)

#title-slide()

== Results
The logos are in the footer, after
the UFAC one.
```





1.4 Language

The theme speaks Portuguese by default (`lang: "pt-br"`): Exercício, Exemplo, Parte. `lang: "en"` or `"es"` switches the text language and those three names, and each name can still be overridden.

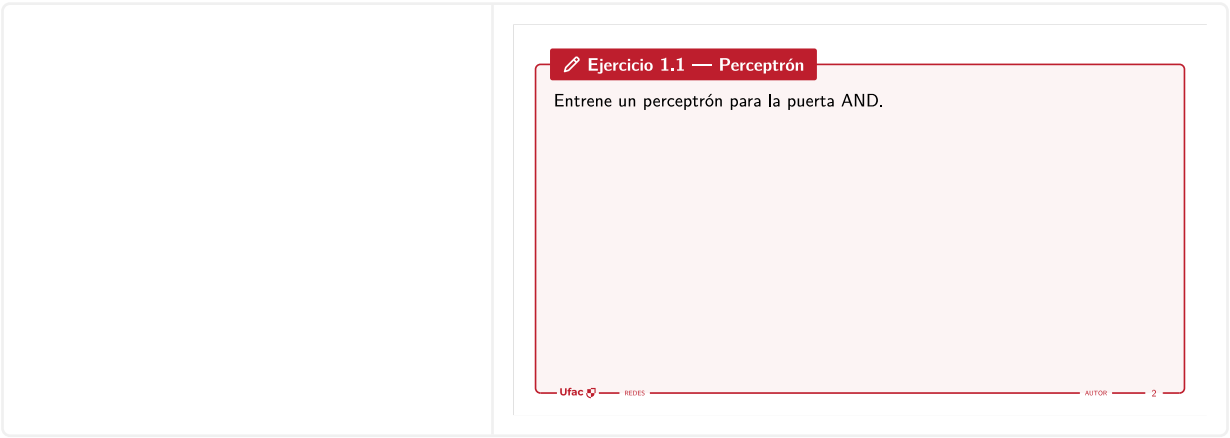
```typst
#show: ufac-theme.with(
  lang: "es",
  part-name: [Unidad],
  config-info(title: [Redes],
author: [Autor]),
)

= Redes multicapa

== Perceptrón <exercise>

Entrene un perceptrón para la
puerta AND.
```





1.5 Dependencies

The versions below are read from the sources of the package, so this table cannot go stale. They are fetched automatically by Typst. touying 0.8 requires Typst 0.15; this version stays on 0.7.4, which also works with Typst 0.14. codly is optional: a deck that wants it imports codly 2 itself and passes it to the theme (see Section 8.3).

Package	Version	Role
`codly`	`2.0.0`	
`iconify`	`0.5.3`	the icons of a collection the deck provides (`provide-icons`); the theme's own icons are shipped with it
`touying`	`0.7.4`	the slide engine; the deck imports it too, with the same version

Besides the packages, the theme needs **Typst 0.14.0 or later** and the font **New Computer Modern Sans** installed on the system (it is not bundled with Typst, unlike **New Computer Modern Math**, used for the equations). Building this manual also takes tidy 0.4.3, and the test suite runs on Tytanic 0.4.1.

2 Slides and headings

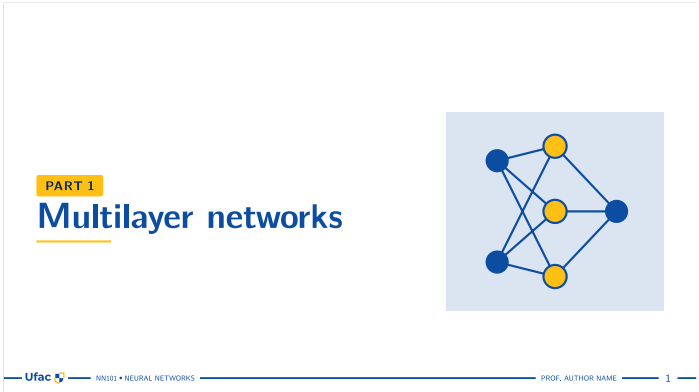
Headings drive the deck: no function call is needed for the ordinary slides.

Markup	Effect
`= Title`	section slide “Part N: Title”; what follows the heading goes to the right
`== Title`	new slide with a blue header, fitted to the width
`== Title <exercise>`	exercise slide: red pill with a pencil, “Exercise 1.N — Title”, numbered automatically
`== Title <example>`	example slide: green pill with a beaker, “Example 1.N — Title”, with its own counter
`=== Subtitle`	blue pill; no new slide; repeated at the top of every continuation page
`==== Subtitle`	the same pill, not repeated
`---`	alone on a line: next page of the same slide (same header, same number)

2.1 Sections

Every `=` heading opens a section slide, as the cover in white: “PART N” in the yellow tag (the name follows the language of the deck, or `ufac-theme(part-name:)`), the title in blue and the yellow rule. The content right after the `=`, typically an image, sits on the right, at the page margin from the edge.

```
= Multilayer networks
#image("figure.svg")
```



A photo or a figure with a background looks better flush with the edge: `section-margin: false` in `ufac-theme` puts it there on every section slide, and a length sets another distance.

```
#show: ufac-theme.with(
  lang: "en",
  section-margin: false,   //
flush with the edge
  config-info(
    title: [Networks],
    author: [Author Name],
  ),
)

= Multilayer networks
#image("network.svg")
```



2.2 Outline

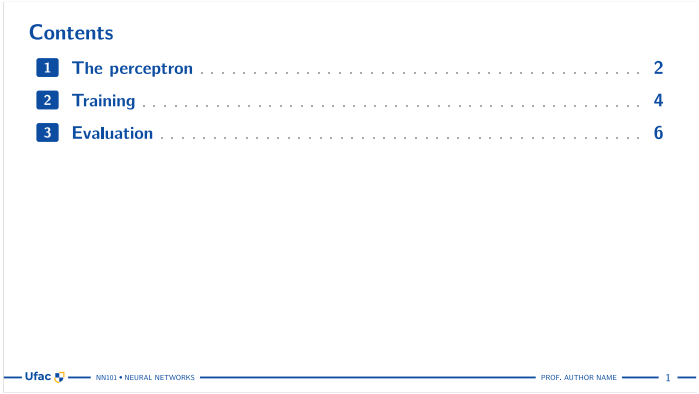
`#outline()` lists the parts of the deck, the `= Title` headings, with their numbers in small pills, the titles in bold blue and the number of the slide where each one starts, the number its footer shows; every line is a link to that slide. It usually takes a slide of its own, right after the cover. In a deck it stands for Typst's `outline`, which is still there as `std.outline`.

```
== Contents
#outline()

= The perceptron
== One neuron
A weighted sum and a threshold.

= Training
== Gradient descent
Small steps against the gradient.

= Evaluation
== Metrics
Accuracy, precision and recall.
```



outline()

```
outline(
) -> content
```

Outline of the deck: its parts (the `= Title` headings, the "Part N" of the section slides), one per line, with the number in a small pill, the title in bold blue and, after a dotted leader, the number of the section slide (the slide number of the footer, which skips the cover); every line is a link to that slide. In a deck it takes the place of Typst's `outline`, which is still `std.outline`. It goes on a slide of its own, usually the first after the cover:

```
== Contents
#outline()
```

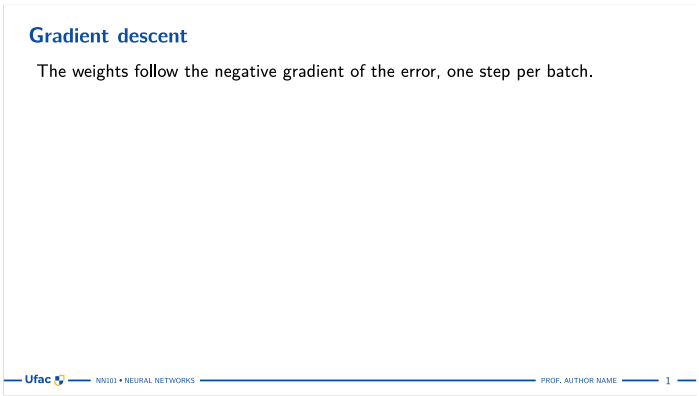
2.3 Slides and continuations

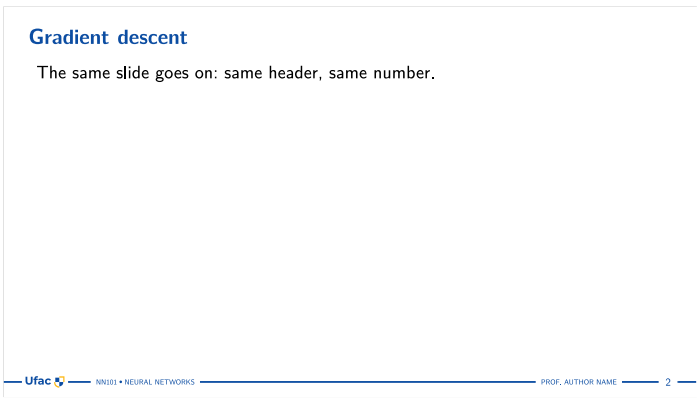
```
== Gradient descent

The weights follow the negative
gradient of the error,
one step per batch.

---

The same slide goes on: same
header, same number.
```





Caution

Touying breaks the page at *any* top-level em dash, so an em dash next to inline markup, as in `Text – *bold*`, also starts a new page. Use an en dash or wrap the paragraph in `#[ ]`.

2.4 Exercises and examples

A label on the `==` heading turns the slide into the box of an exercise (red) or an example (green), as in a printed handout: the box fills the slide, the title goes on a badge over its top border, with the icon of the kind (a pencil for exercises, a beaker for examples), the name, the number and, after a dash, the title, and the footer of the slide closes the box at the bottom, in the colour of the item. Any suffix gives the same slide and makes the label unique, so the slide can be cited: `@exercise-ogata` becomes "Exercise 1.N", bold, in the color of the kind and linked to the slide. `#exercise-slide` and `#example-slide` are the function forms, with `color` and `icon` options; they cannot be cited.

Inside the box, only what has no style of its own takes the colour of the item: the bold text, the markers, the arrows, the highlights, the numbers of the lists, the inline code, the links and the pills of the `===` and `====` subtitles. A code block, a callout and an `emph-box` keep their own look.

`#solution` (in an example) and `#answer` (in an exercise), alone on a line like `#pause`, open the solution: a dashed line and "Solution." / "Answer." in bold. With a `#pause` right before or after the marker, the solution appears on the next subslide. An exercise or example can take several pages: a content longer than one page goes on to the next one by itself, in the same box, with the same number and the `===` subtitle; a `---` or a `#pagebreak()` breaks the page by hand. The item goes on until a slide of another kind.

The icon is the name of an `icon`, any content (an image) or `none`, which leaves only the text. It is centered on the capitals of the text, whatever its height. `ufac-theme(exercise-icon:, example-icon:)` changes it in the whole deck; `icon:` changes it on one slide.

```
== Ogata 4.6 <exercise-ogata>

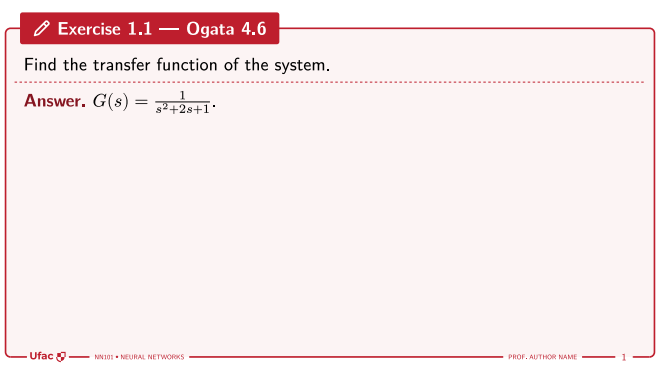
Find the transfer function of the
system.

#answer
$G(s) = 1 / (s^2 + 2s + 1)$.

== XOR network <example>

Can a single neuron learn XOR?

#pause
#solution
```



```
No: two hidden neurons are enough.

#exercise-slide(title: [Haykin
4.2], color: colors.primary, icon:
"terminal")[
  The function form, with another
color and icon.
]

#example-slide(title:
[Perceptron], icon: none)[
  Without icon: `icon: none`.
]

== Review

Solve @exercise-ogata before the
next class.
```

Example 1.1 — XOR network

Can a single neuron learn XOR?

Solution. No: two hidden neurons are enough.

Example 1.1 — XOR network

Can a single neuron learn XOR?

Solution. No: two hidden neurons are enough.

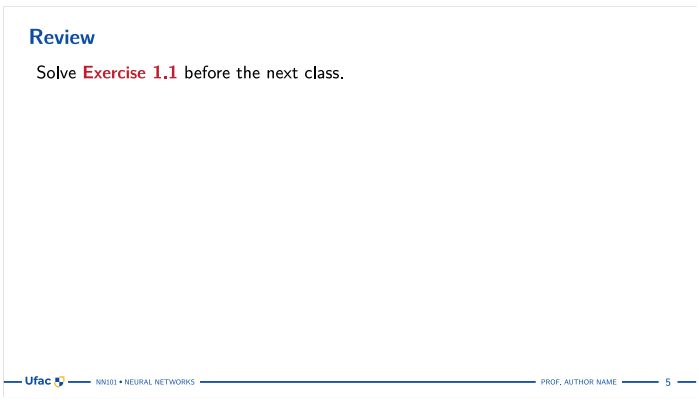
Ufac NN101 • NEURAL NETWORKS PROF. AUTHOR NAME 2

Exercise 1.2 — Haykin 4.2

The function form, with another color and icon.

Example 1.2 — Perceptron

Without icon: `icon: none`.



exercise-slide()

```
exercise-slide(
  title: [] content
  color: auto auto color
  icon: auto auto none str content
  body content
) -> content
```

Exercise slide: the header becomes a pill with a pencil, "Exercise 1.N — Title", numbered automatically. The function form of `== Title <exercise>`. Only the heading form can carry a unique label (`== Title <exercise-ogata>`) and be cited with `@exercise-ogata`, which shows "Exercise 1.N" linked to the slide.

```
#exercise-slide(title: [Ogata 4.6])[Find the transfer function.]
#exercise-slide(title: [Lab], color: colors.primary, icon: "tools")[Another color and
icon.]
```

Argument

`title: []` — content

Text after the number, following a dash; empty omits the dash.

Argument

`color: auto` — auto | color

Color of the pill; `auto` is red (`tertiary`).

Argument

`icon: auto` — auto | none | str | content

Icon before the name: the name of an `icon`, any content (an image) or `none` for no icon; `auto` is the icon of the theme, `ufac-theme(exercise-icon:)` (a pencil).

Argument

body — content

Content of the slide; `---` inside it continues the same exercise.

`example-slide()`

```
example-slide(
  title: [] content
  color: auto auto color
  icon: auto auto none str content
  body content
) -> content
```

Example slide: a pill with a beaker, "Example 1.N — Title", with a counter independent from the exercises. The function form of `== Title <example>`; see `exercise-slide()`.

```
#example-slide(title: [XOR network])[Two hidden neurons are enough.]
```

Argument

`title: []` — `content`

Text after the number, following a dash; empty omits the dash.

Argument

`color: auto` — `auto` | `color`

Color of the pill; `auto` is green (`quaternary`).

Argument

`icon: auto` — `auto` | `none` | `str` | `content`

Icon before the name: the name of an `icon`, any content (an image) or `none` for no icon; `auto` is the icon of the theme, `ufac-theme(example-icon:)` (a beaker).

Argument

`body` — `content`

Content of the slide.

`solution` `content`

The marker of the start of the solution of an example, alone on its line like Touying's `#pause`: what comes before it is the statement and what comes after it the solution, opened by a dashed line and "Solução." / "Solution." in bold. With a `#pause` right before or right after it, the solution appears on the next subslide, opening included.

```
== The state of water <example>
A script that reads the temperature and prints the state of the water.
#pause
#solution
Every condition is an `if`: …
```

`answer` `content`

The marker of the start of the answer of an exercise: the same as `solution`, opened by "Resposta." / "Answer.".

2.5 Subtitles

A `===` subtitle is repeated on the pages that continue the slide, unless the continuation starts with another `===`. That holds for `---`, for `#pagebreak()` and for content that overflows by itself.

```
== Activation functions

=== Sigmoid

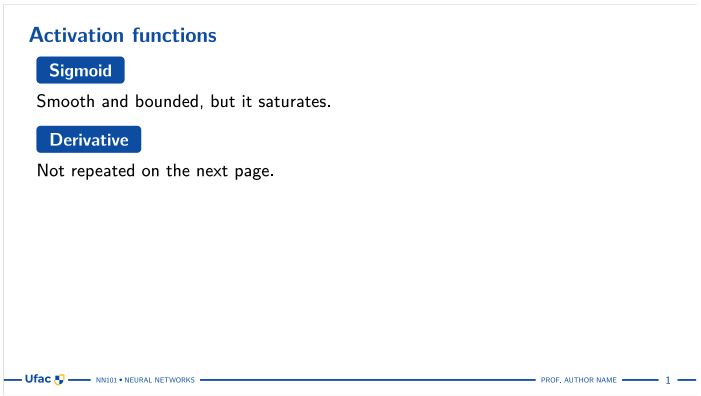
Smooth and bounded, but it
saturates.

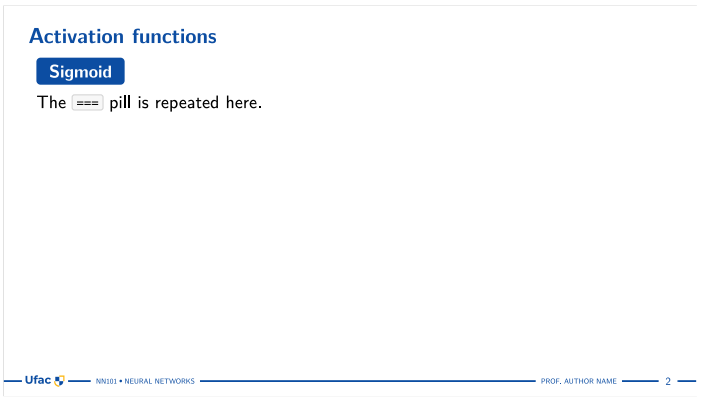
==== Derivative

Not repeated on the next page.

---

The `===` pill is repeated here.
```

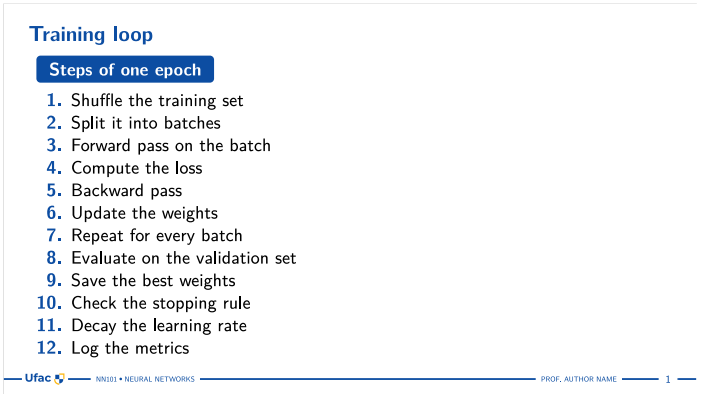


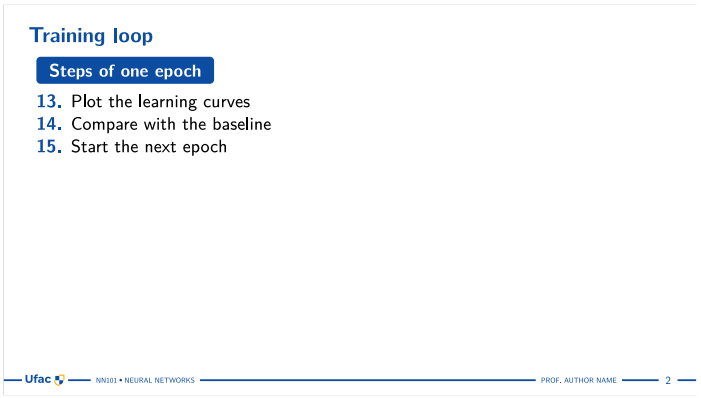



```
== Training loop

=== Steps of one epoch

+ Shuffle the training set
+ Split it into batches
+ Forward pass on the batch
+ Compute the loss
+ Backward pass
+ Update the weights
+ Repeat for every batch
+ Evaluate on the validation set
+ Save the best weights
+ Check the stopping rule
+ Decay the learning rate
+ Log the metrics
+ Plot the learning curves
+ Compare with the baseline
+ Start the next epoch
```





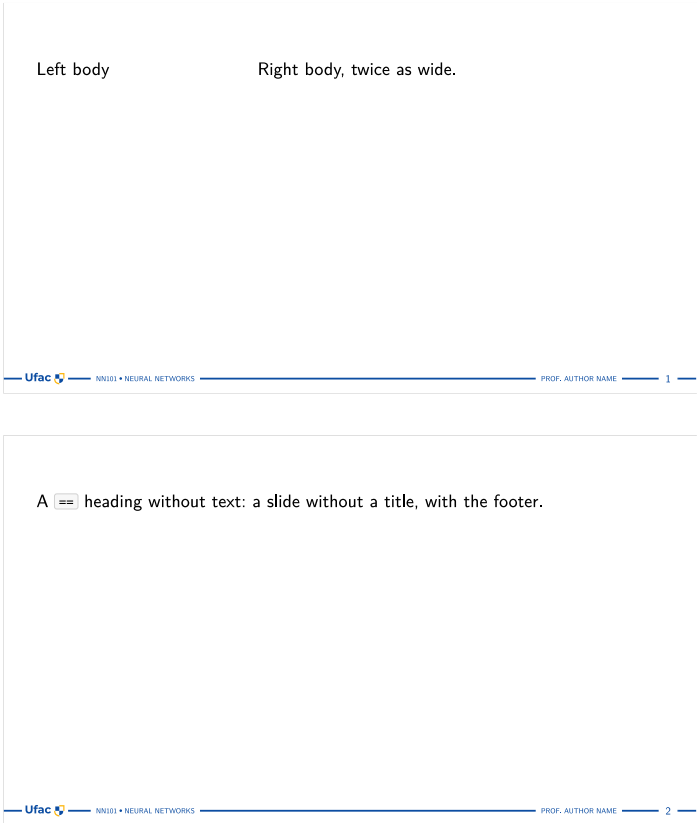
Only a top-level `===` is repeated; inside `#[ ]`, a column or a box the pill stays in the flow. `config-common(breakable: false)` makes Touying warn about overflow instead.

2.6 Slide functions

`#slide` is the slide that every `==` opens; calling it gives access to Touying's arguments, such as `composer`. A `==` heading without text gives a slide without a title, with the footer. `#title-slide()` is the cover, shown in Section 1; `#title-slide[..]` adds content below its titles.

```
#slide(composer: (1fr, 2fr))[
  Left body
][
  Right body, twice as wide.
]

==
A `==` heading without text: a
slide without a title, with the
footer.
```



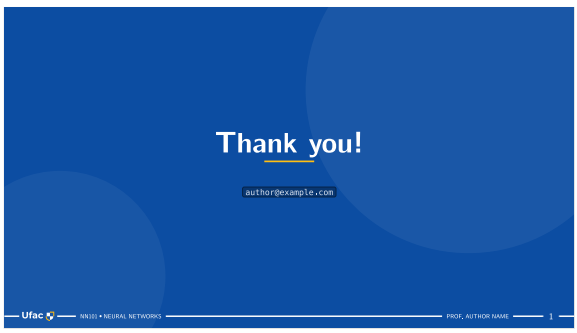
The closing slide: `#ending-slide`

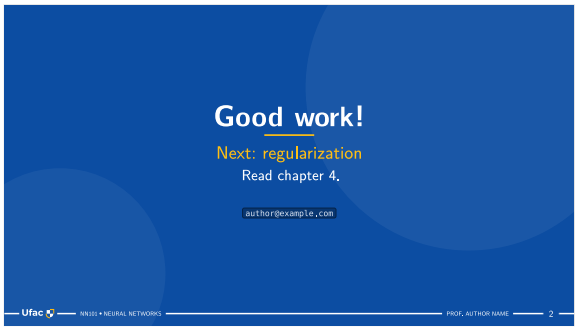
`#ending-slide` closes the deck on the blue page of the cover: a large title with the yellow rule, a subtitle in yellow, a free body and the contact of `config-info` right below them, and the footer of the slides in white, with the logos. Without a title it says "Obrigado!" / "Thank you!" in the language of the deck. The labels `<empty-slide>` and `<empty-slide-primary>` of 0.2.0 are gone; Touying's own `#empty-slide` (no header nor footer) is still there for whoever imports Touying.

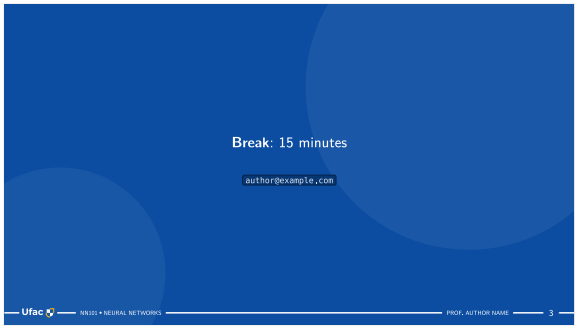
```
#ending-slide()

#ending-slide[Good work!][Next:
regularization][
  Read chapter 4.
]

#ending-slide(title: none)[
  *Break*: 15 minutes
]
```







slide()

```
slide(
  config: (:) dictionary
  repeat: auto auto int
  setting: body => body function
  composer: auto auto array function
  ..bodies content
) -> content
```

Default slide of the theme, with header and footer. Every `==` heading opens one, so it is rarely called by hand; the explicit form gives access to Touying's arguments.

A top-level `===` subtitle is repeated at the top of every continuation page of the slide, whether the page comes from `---`, from `#pagebreak()` or from content that overflows by itself, unless the continuation starts with another `===`. A subtitle nested in `#[ ]`, a column or a box is not repeated.

A slide without a title is a `==` heading without text: it keeps the footer. The closing slide is `ending-slide()`.

```
#slide(composer: (1fr, 2fr))[Left][Right]
```

Argument

`config: (:)` `dictionary`

Slide configuration: a `config-xxx` dictionary (merge several with `utils.merge-dicts`).

Argument

`repeat: auto` `auto` | `int`

Number of subslides; `auto` lets Touying count the animations.

Argument

`setting: body => body` `function`

Set/show rules applied to the slide body.

Argument

`composer: auto` `auto` | `array` | `function`

Layout of the bodies, e.g. `(1fr, 2fr)`, as in Touying's `side-by-side`.

Argument

`..bodies` `content`

The slide bodies.

ending-slide()

```
ending-slide(
  title: auto auto none content
  subtitle: auto auto none content
  contact: auto auto none content
  ..args content
) -> content
```

Closing slide (the name of the templates of Tylina): the blue page of the cover, with a large title with the yellow rule under it, a subtitle in yellow, a free body and the contact of `config-info` right below them, centered; and the footer of the slides in white (the logos, the title, the authors and the slide number). Without a title it says "Obrigado!" / "Thank you!" in the language of the deck; `title: none` removes the title and the rule.

Two forms: up to three contents in order, `[title][subtitle][body]`, or the named `title:` and `subtitle:` with the content as the body.

```
#ending-slide()
#ending-slide[Good work!][Next lecture: functions][Read chapter 3.]
#ending-slide(title: [Questions?], subtitle: [Lab 4 on Friday])[*See you!*]
```

Argument

`title: auto` `auto` | `none` | `content`

The title; `auto` is "Obrigado!" in the language of the deck, `none` removes it.

Argument

`subtitle: auto` `auto` | `none` | `content`

The subtitle, in yellow; `auto` and `none` give none.

Argument

`contact: auto` `auto` | `none` | `content`

The contact under the content; `auto` is the one of `config-info(contact:)`, `none` hides it.

Argument

`..args` `content`

Up to three contents, `[title][subtitle][body]`, or the body alone with `title:` or `subtitle:`.

new-section-slide()

```
new-section-slide(
  config: (:) dictionary
  level: 1 int
  numbered: true bool
  body content
) -> content
```

Section slide, created by every `= Title`: on the left, "PART N" in the yellow tag, the title in blue and the yellow rule, as on the cover; on the right, the content right after the `=` (typically an `#image`), at the page margin from the edge. `ufac-theme(section-margin: false)` puts it flush with the edge on every section slide, for photos and figures with a background. Not meant to be called by hand.

Argument

`config: (:)` `dictionary`

Slide configuration, see `slide.config`.

Argument

`level: 1` `int`

Heading level, passed by Touying.

Argument

`numbered: true` `bool`

Passed by Touying; the part is always numbered.

Argument

`body` `content`

Content that follows the `=` heading.

3 Inline formatting and shorthands

Most emphases have two forms: a function and a **shorthand**, a short piece of markup that the theme rewrites into the same function. The shorthands take plain text only (no markup inside) and none of them applies inside `raw`.

Function	Shorthand	Result
`#strong[text]`	`*text*`	black bold
`#emph[text]`	`_text_`	black italic
`#alert[text]`, `#primary[text]`	`~~text~~`	blue bold
`#highlight[text]`	`==text==`	light-yellow background
`#underline[text]`	`~text~`	strong yellow underline
`#secondary`, `#tertiary`, `#quaternary`		bold in yellow, red, green
`#emph-box[..]`	`> text`	box, Section 5
`$ #eq-box[$..$] $`	`> $ .. $`	equation box, Section 5
`#quote-box[..]`	`"text"`	quotation, Section 5
`#arrows[..]`	`-> item`	arrow list, Section 4

3.1 Bold, italic and alert

```
Black *bold* and _italic_.

Blue #alert[alert], the same as
~~this shorthand~~.
```

Black **bold** and *italic*.

Blue **alert**, the same as **this shorthand**.

3.2 Highlight and underline

The `==text==` shorthand cannot open a line (there it is a heading) and its text neither starts nor ends with a space; `#highlight` has no such limits and takes another fill, such as the `-lighter` tones of the palette.

```
A ==highlighted== word and a
#highlight(fill: colors.quaternary-
lighter)[green one].

A ~strong underline~ and
#underline[the function form]; `Fig.~1`
keeps
its non-breaking space.
```

A highlighted word and a green one.

A strong underline and the function form `Fig.~1` keeps its non-breaking space.

3.3 Code chips

Inline `raw` is a gray chip. Inside a title or a box it follows the color of the container, and inside a pill it is inverted.

```
Call `fit(x, y)` first.

#emph-box(color: colors.tertiary)[
  Never call `predict` before `fit`.
]
```

Call `fit(x, y)` first.

Never call `predict` before `fit`.

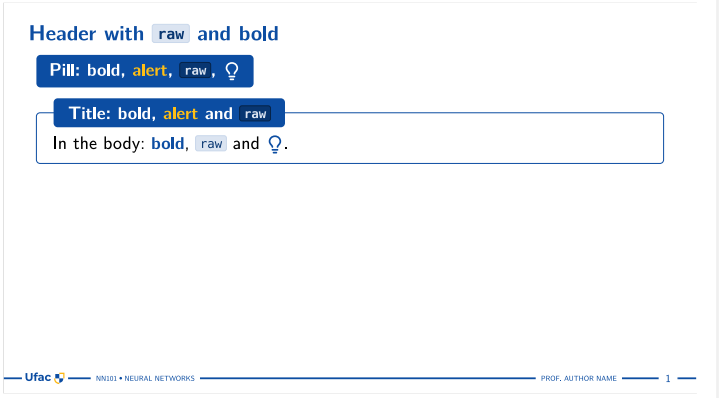
3.4 Colors by context

Bold text, alerts, highlights, underlines, arrows, chips, links, icons and the pills of the `===` and `====` subtitles read the color of what contains them: the box color inside an `emph-box`, a callout, an exercise or an example and a light cell of a table; white or yellow inside a pill, where the pill color would hide them. What has a style of its own keeps it: a code block, and a box inside another box, which brings its own color.

```
== Header with `raw` and *bold*

=== Pill: *bold*, ~~alert~~, `raw`, #icon("tip")

#emph-box(title: [Title: *bold*, ~~alert~~ and `raw`])[
  In the body: *bold*, `raw` and #icon("tip").
]
```



3.5 Colored emphases

Bold text named after the color families of Section 7. A typical use is a label that opens a sentence.

`primary()`

```
primary(
  body content
) -> content
```

Bold text in UFAC blue: the same as `#alert` (Touying's method) and as the `~~text~~` shorthand. Inside a pill (a `===` subtitle, an exercise header, the title of an `emph-box()`) it turns UFAC yellow, where blue would vanish.

```
A #primary[definition], the same as ~~this shorthand~~.
```

A **definition**, the same as **this shorthand**.

Argument

body `content`

Text to emphasize.

`secondary()`

```
secondary(
  body content
) -> content
```

Bold text in UFAC yellow. It is weak on white: meant for short labels and dark backgrounds.

```
A #secondary[remark] in yellow.
```

A **remark** in yellow.

Argument

body `content`

Text to emphasize.

`tertiary()`

```
tertiary(
  body content
) -> content
```

Bold text in red, for warnings and problems.

```
#tertiary[Caution:] the gradient vanishes.
```

Caution: the gradient vanishes.

Argument

body `content`

Text to emphasize.

`quaternary()`

```
quaternary(
  body content
) -> content
```

Bold text in green, for advantages and correct answers.

```
#quaternary[Correct:] the series converges.
```

Correct: the series converges.

Argument

body `content`

Text to emphasize.

3.6 Escapes

A backslash on the sign keeps a shorthand literal. An escaped `>` is only recognized when a word follows it.

```
\==not highlighted== and \~not
underlined\~

\> not a box

-\> not a list

\"not a quotation\"
```

==not highlighted== and ~not underlined~

> not a box

-> not a list

"not a quotation"

4 Lists and terms

4.1 Arrow lists

Consecutive lines starting with `->` become one list with a blue arrow. The list has to start the paragraph: a blank line comes before the first `->`. An arrow in the middle of a line and the math arrow are left alone.

```
-> consequence
-> another consequence

Here a -> b stays text, and $a -> b$ is
math.
```

→ consequence
→ another consequence

Here a -> b stays text, and $a \to b$ is math.

`arrows()`

```
arrows(
  color: colors.primary  color
  body  content
) -> content
```

Arrow lists with the arrow in another color, for everything inside the body. The `-> item` shorthand itself needs no function: consecutive lines starting with `->` become one list with a blue arrow (the box color inside an `emph-box()`). The list must start the paragraph, and `-\> item` escapes it.

```
#arrows(color: colors.quaternary)[
  -> green here
  -> and here
]
```

→ green here
→ and here

Argument

`color: colors.primary` `color`

Color of the arrows.

Argument

body `content`

Content with `-> item` paragraphs.

4.2 Bullet lists

Three levels of yellow markers: a filled square, a rotated hollow square and a small circle.

```
- first level
  - second level
    - third level
- first level again
```

- first level
  - second level
    - third level
- first level again

4.3 Numbered lists

Numbers are always blue and bold: `1.`, `a)`, `i.` by level. `#set enum(numbering: "a)")` starts at `a)` instead and keeps the colors. Typst only knows `+` and `1.` as enum markers in markup, so `a)` at the start of a line is text.

```
+ first
  + nested
    + deeper
+ second

#[
  #set enum(numbering: "a)")
  + starts at a)
    + then i.
]
```

1. first
   a) nested
      i. deeper
2. second

a) starts at a)
   i. then i.

4.4 Terms

```
/ Epoch: one pass over the training set.
/ Batch: the samples of one update.
```

Epoch: one pass over the training set.
Batch: the samples of one update.

5 Boxes and quotes

5.1 Emphasis boxes

`emph-box()`

```
emph-box(
  color: colors.primary  color
  title: none  none  content
  icon: none  none  str  content
  body  content
) -> content
```

Emphasis box. Without a title it has a thin border; with a title, a badge sits over the top border. Inside the box the `-` markers, the `->` arrow, `*bold*`, `==highlight==` and `#underline` take the box color; an explicit `#arrows(color:)`, `#highlight(fill:)` or `#underline(stroke:)` wins. Colors by convention: primary for definitions, quaternary for advantages, tertiary for problems, secondary for remarks.

```
#emph-box(color: colors.quaternary, title:
[Pros])[
  - Fast and *simple*
  - Works with ==highlights==
]
#emph-box(color: colors.tertiary, icon:
"alert", title: [Caution])[
  An icon before the title.
]
```

Pros

- Fast and **simple**
- Works with highlights

⚠ Caution

An icon before the title.

Shorthand, always in the primary color: a paragraph whose lines start with `>`; a first line `> == Title` gives the title. Only inline content fits the shorthand. For a framed equation see `eq-box()`.

Argument

`color: colors.primary` — `color`

Color of the border and badge, and of the bold text, markers, arrows and highlights inside.

Argument

`title: none` — `none` | `content`

Badge over the top border; `none` gives the thin-border box (unless there is an `icon`).

Argument

`icon: none` — `none` | `str` | `content`

Icon at the start of the badge, before the title: the name of an `icon()`, any content (an image, a symbol) or `none`. It is centered on the capitals of the title, as in the exercise and example pills; without a title it is the whole badge. The badge has the same height with or without an icon.

Argument

`body` — `content`

Content of the box.

```
#emph-box[A definition, in the primary
color.]

#emph-box(color: colors.tertiary, title:
[Problem])[
  The gradient *vanishes* and ~stalls~.

  -> the arrow is red too
]

#emph-box(color: colors.quaternary, icon:
"light-bulb", title: [Tip])[
  An `icon` before the title, as in the
exercise pills.
]
```

A definition, in the primary color.

Problem

The gradient **vanishes** and stalls.

→ the arrow is red too

Tip

An `icon` before the title, as in the exercise pills.

5.2 The box shorthand

A paragraph whose lines start with `>` is a box in the primary color; a first line `> == Title` gives the title.

```
> A box without title, with *bold* and
$x^2$.

> == Definition
> A box with a title.
> The second line flows on.
```

A box without title, with **bold** and x^2.

Definition

A box with a title. The second line flows on.

Limits of the shorthand

Only inline content fits: lists and code blocks need `#emph-box`, and `>` before a block equation is an `eq-box`. A loose `>` at the start of a paragraph is read as a box: write `$a > b$`, or escape it (Section 3.6).

5.3 Equation boxes

eq-box()

```
eq-box(
  color: auto  auto  color
  body  content
) -> content
```

Equation box: a light frame around a key equation (the `key-eq` helper of the lecture decks). It goes **inside** the equation, like the `boxeq` of the bookly package, so the equation keeps its centering, spacing and numbering, and a part of an equation can be framed too. The body is written as math, between dollar signs; it is set in display style whatever the spacing around it. A box around the whole equation is padded generously, a box around a part of one tightly, and in both cases the frame holds tall fractions and the limits of sums.

It also works in a line of text or in an inline equation: there the body is in text style and the box does not change the line spacing.

```
The output #eq-box[$y = phi(v)$] and the
mean #eq-box[$1/N sum_i y_i$].
```

The output $y = \varphi(v)$ and the mean $\frac{1}{N}\sum_i y_i$.

```
$ #eq-box[$v = bold(w)^top bold(x)$] $

$ y = #eq-box(color: colors.quaternary)
[$phi(v)$] + b $
```

$$v = \boldsymbol{w}^\top \boldsymbol{x}$$

$$y = \boxed{\varphi(v)} + b$$

Shorthand, always in the primary color: `>` in front of a block equation, on the same line or alone on the line before it.

```
> $ E = 1/(2N) sum_i (y_i - hat(y)_i)^2 $
```

$$E = \frac{1}{2N} \sum_i (y_i - \hat{y}_i)^2$$

Argument

`color: auto` — `auto` | `color`

Base color of the fill and border; `auto` is blue, or the box color inside an `emph-box()`.

Argument

body — `content`

The equation, as math: `[$x = 2$]` or `($x = 2$)`.

Why the dollar signs inside

A content block inside an equation is markup, not math: `#eq-box[x = 2]` would be the upright text "x=2", and an underscore or an asterisk in it would not even parse. `eq-box` refuses such a body with a hint. `> $x$`, with an inline equation, is an ordinary box: the shorthand takes a block equation.

5.4 Callouts

A warning, a tip, a remark or a note, each with its colour, icon and title in the language of the deck. The default style is the box of a printed handout, with a bar on the left and the title opening the text; `style: "badge"` gives an emphasis box with the title on a badge.

`callout()`

```
callout(
  kind str
  style: "bar" str
  title: auto auto none content
  color: auto auto color
  icon: auto auto none str content
  body content
) -> content
```

Callout: a box with the colour, the icon and the title of its kind, the title in the language of the deck. The default style, `"bar"`, is the box of a printed handout: a bar on the left, a light fill, the icon and the title in bold opening the text ("**Tip.** …", "**Warning!** …"); `"badge"` is an `emph-box()` with the title on a badge over the border. Inside it, as in a box, the bold text, the markers and the arrows take its colour; a code block keeps its own look.

```
#callout("warning")[`x*y` with two vectors is an error.]
#callout("tip")[The up arrow recalls a command.]
#callout("info", style: "badge")[The style with the badge.]
```

Warning! `x*y` with two vectors is an error.

Tip. The up arrow recalls a command.

Remark

The style with the badge.

Argument

`kind` — `str`

`"warning"` (red), `"tip"` (green), `"info"` (blue) or `"note"` (gray).

Argument

`style: "bar"` — `str`

`"bar"` (a bar on the left and the title opening the text) or `"badge"` (the title on a badge over the border).

Argument

`title: auto` — `auto` | `none` | `content`

The title; `auto` is the name of the kind in the language of the deck (“Warning” / “Atenção”; in the style `"bar"` with its punctuation, “Warning!”, “¡Atención!”, “Tip.”); `none` leaves the icon alone.

Argument

`color: auto` — `auto` | `color`

The colour; `auto` is the one of the kind.

Argument

`icon: auto` — `auto` | `none` | `str` | `content`

The icon: a name of `icon`, content or `none`; `auto` is the one of the kind.

Argument

`body` — `content`

The text.

5.5 Tags

The yellow tag of the cover and of the section slides is an element of its own, for the text, the titles and the tables: a short label in spaced capitals, on any colour.

`tag()`

```
tag(
  color: colors.secondary  color
  size: 0.6em  length
  body  content
) -> content
```

Tag: capitals, spaced and bold, on a colour, with rounded corners; by default UFAC yellow with the text in dark blue, as on the cover (“CURSO DE EXTENSÃO”) and on the section slides (“PARTE 2”). The text is dark blue or white, whichever contrasts more with the colour. The vertical padding is an `outset`, as in `eq-box()`: in a line of text the tag does not change the height of the line.

```
-> `plot3` #tag[new]
-> #tag(color: colors.tertiary)[exam] on
Friday
-> #tag(color: colors.quaternary-light)
[optional]
```

→ plot3 NEW
→ EXAM on Friday
→ OPTIONAL

Argument

`color: colors.secondary` `color`

The colour of the tag.

Argument

`size: 0.6em` `length`

The size of the text, relative to the text around it.

Argument

body `content`

The text, in capitals.

5.6 Quotations

`quote-box()`

```
quote-box(
  color: colors.secondary  color
  body  content
) -> content
```

Quotation box: a vertical bar on the left. Typst’s `#quote(attribution: [Author])[…]` is shown as a `quote-box` too, with the attribution on the right.

```
#quote-box(color: colors.primary)[
  All models are wrong, but some are
useful.
]
```

> All models are wrong, but some are useful.

Shorthand: a paragraph made only of `"quoted"` runs, one per line; each run becomes a box and the colors cycle secondary, primary, quaternary, tertiary. Quotes inside a sentence are left alone.

Argument

`color: colors.secondary` `color`

Color of the bar.

Argument

body `content`

The quotation.

```
"All models are wrong"
"but some are useful"

A sentence with a "quotation" inside.

#quote(attribution: [George Box])[
  All models are wrong.
]
```

> All models are wrong

> but some are useful

A sentence with a “quotation” inside.

> All models are wrong.
>
> — George Box

6 Columns

cols()

```
cols(
  columns: auto  auto  array
  gutter: 1em  length
  divider: false  bool
  ..bodies  content
) -> content
```

Columns: a thin wrapper over Touying's `cols` (a `grid`), with an optional divider. Touying's own `side-by-side` (no divider) is still available.

```
#cols(divider: true, columns: (1fr, 2fr))[
  Left
][
  -> the shorthands work
  -> inside a column
]
```

Left

→ the shorthands work
→ inside a column

Argument

`columns: auto` — `auto` | `array`

Column widths, as Touying accepts them: `auto` (equal widths) or an array such as `(1fr, 2fr)`.

Argument

`gutter: 1em` — `length`

Space between the columns.

Argument

`divider: false` — `bool`

A blue vertical bar (2.2pt) between the columns; the gutter is then used as inset.

Argument

`..bodies` — `content`

One content block per column.

```
#cols[Left][Middle][Right]
```

Left Middle Right

Touying's `side-by-side` (no divider) is still available, and `#slide(composer: (1fr, 2fr))[..][..]` lays out the bodies of a whole slide.

7 Colors and logo

Five families of seven tones. The light tones serve as fills of highlights and boxes, the dark ones as text on white.

	`-darkest`	`-darker`	`-dark`	base	`-light`	`-lighter`	`-lightest`
`primary`	#031329	#052349	#083671	#0C4DA2	#5582BE	#9EB8DA	#DBE4F1
`secondary`	#403005	#735609	#B2860E	#FFBF14	#FFD25A	#FFE5A1	#FFF5DC
`tertiary`	#30070B	#560D14	#851520	#BE1E2D	#D2626C	#E5A5AB	#F5DDE0
`quaternary`	#022C20	#044F3A	#067B5A	#09B081	#53C8A7	#9DDFCD	#DAF3EC
`neutral`	#000000	#2B2B2B	#555555	#808080	#AAAAAA	#D5D5D5	#FFFFFF

The tones are derived from the base in RGB: `darken(75%)`, `darken(55%)`, `darken(30%)`, the base, `lighten(30%)`, `lighten(60%)` and `lighten(85%)`. The neutral family follows Touying's default instead, from black to white.

```
#highlight(fill: colors.tertiary-lighter)
[red fill]
and #text(fill: colors.primary-dark)[dark
blue text].
```

red fill and dark blue text.

`colors` `dictionary`

The theme palette: the official UFAC colors plus a neutral gray, named after Touying's `config-colors` keys (`primary`, `secondary`, `tertiary`, `neutral`) plus `quaternary`. Each family has 7 tones, `<family>-darkest`, `-darker`, `-dark`, the base, `-light`, `-lighter` and `-lightest`, so `colors.primary-lighter`, `colors.quaternary-dark` etc. exist. The whole dictionary is passed to `config-colors`, hence the same keys are available as `self.colors.*`.

Family	Base	Role
`primary`	`#0C4DA2`	titles, `->` markers, tables, `#alert`/`#primary`, pills
`secondary`	`#FFBF14`	`-` markers, `#secondary`, quotes; `-lighter` is the highlight
`tertiary`	`#BE1E2D`	`#tertiary`, exercises
`quaternary`	`#09B081`	`#quaternary`, examples
`neutral`	`#808080`	`-darkest` is the bold/italic text, `-lightest` the background

The UFAC logo comes with the theme, as its SVG (`ufac-logo`) and as an image in the colours the theme uses (`ufac-logo-image`): blue lettering with the yellow shield in the footer, white lettering on the cover, and all white on a background where the yellow would not stand out, such as the cover of a handout that goes with the slides.

`ufac-logo` `str`

Official UFAC logo as an SVG string (yellow shield and blue lettering). `ufac-logo-image()` draws it, also in other colours (the footer, blue; the cover, white with the yellow shield).

The logo belongs to the Universidade Federal do Acre and is not covered by the license of the package.

```
#image(bytes(ufac-logo), height: 2em)
```

Ufac

ufac-logo-image()

```
ufac-logo-image(
  height: 0.64em  length
  color: auto  auto  color
  shield: auto  auto  color
) -> content
```

The UFAC logo as an image, in the colours the theme uses: blue lettering with the yellow shield, as in the footer of the slides; the lettering in another colour, as on the cover (`color: white`, the shield still yellow); or all in one colour (`color: white, shield: white`), on a background where the yellow would not stand out. For a document that goes with the slides, such as a handout, or a logo next to it in `ufac-theme(logos:)`.

```
#ufac-logo-image(height: 2em)
#box(fill: colors.primary, inset: 0.4em,
  ufac-logo-image(height: 2em, color:
white))
#box(fill: colors.primary, inset: 0.4em,
  ufac-logo-image(height: 2em, color:
white, shield: white))
```



Argument

`height: 0.64em` — `length`

Height of the logo; `0.64em` is its height in the footer.

Argument

`color: auto` — `auto` | `color`

Colour of the lettering, the blue `#0c4da2` of the logo; `auto` keeps it.

Argument

`shield: auto` — `auto` | `color`

Colour of the shield, the yellow `#fdb813` of the logo; `auto` keeps it.

8 Tables, figures, math, code and icons

8.1 Tables

The header row is blue with white text, and formulas in it turn white too. Code chips follow the background, as in the pills and boxes: in the header they are inverted (dark blue with light text). A cell filled with a light tone of a family of `colors` (`table.cell(fill: colors.secondary-lighter)`) is like a box of the family's `-dark` color, which colors its chips and `*bold*`; a cell filled with a dark tone (`colors.primary`, `colors.tertiary-dark`) is like a pill, with white text and inverted chips. White and colors outside the palette change nothing.

```
#table(
  columns: 3,
  table.header([Model], [Layers],
[$eta$]),
  [MLP], [3], [0.1],
  [CNN], [8], [0.01],
)
```

Model	Layers	η
MLP	3	0.1
CNN	8	0.01

8.2 Figures and math

A figure caption shows its body only, without "Figure N". Math uses New Computer Modern Math, and block equations sit closer to the text than Typst's default. For a framed key equation, see `eq-box` in Section 5.

```
#figure(
  rect(width: 60%, height: 3em, fill:
colors.primary-lightest),
  caption: [The caption, without a
number.],
)

The update rule is
$ bold(w) <- bold(w) - eta nabla
E(bold(w)) $
```


The caption, without a number.

The update rule is

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla E(\boldsymbol{w})$$

8.3 Code

Inline `raw` is the chip of Section 3. The theme does not depend on codly: a code block goes in the box of the theme, a light blue fill with a bar on the left and square corners, at 0.75em, with the line numbers on the left (a long line wraps and continues under the code), the language in a white pill with its logo at the top right and the syntax colours of the theme (`ufac-syntax`). The output of a program, a block in `text`, goes in a green box without numbers. A block never splits across pages, and inside a coloured box (an `emph-box`, an exercise) it keeps its look. The languages with a pill are `python`, `typst`, `matlab`, `bash` and `text`; a block in another language has no pill, and `ufac-theme(code-languages:)` adds languages.

````
== Code blocks

```python
def step(w, grad, lr=0.1):
    return w - lr * grad
```

```text
w = 0.25
```

```bash
typst compile slides.typ
```
````



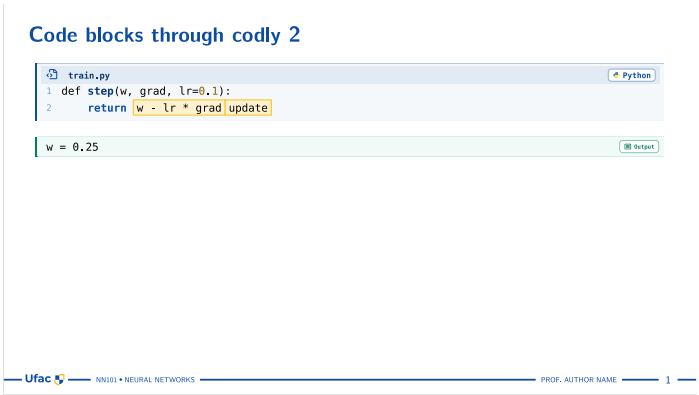
With codly 2, the deck imports it and passes it to the theme, `ufac-theme(codly: codly)`: every block goes through codly with the same look, and codly's own features (a file name, highlights, ranges, `diff`) are at hand through its functions (`codly.new(..)`, `codly.set_(..)`). The theme never imports codly; codly 2 requires Typst 0.15.

````
#import "@preview/codly:2.0.0" as
codly
#show: ufac-theme.with(lang: "en",
codly: codly, config-info(
  title: [Neural Networks],
  short-title: [NN101],
  subtitle: [Training and
evaluation],
  short-subtitle: [Units III and
IV],
  author: [Prof. Author Name],
))

== Code blocks through codly 2

#codly.new(
  ```python
  def step(w, grad, lr=0.1):
      return w - lr * grad
  ```,
  file: "train.py",
  highlights: ((line: 2, start:
12, end: 24, tag: [update]),),
)

```text
w = 0.25
```
````



`code-languages` `dictionary`

The languages the theme knows: the name in the pill of a block and its icon (a name of `icon`, content or `none`). `ufac-theme(code-languages:)` adds languages or replaces these, in the same format; the dictionary of the codly-languages package works too. The output (`text`, `txt`, `output`) is "Saída" / "Output" / "Salida" in the language of the deck (`name: auto`).

`codly-ufac-theme` `dictionary`

The UFAC theme for codly 2, a dictionary in the format of its themes, for `codly.theme(..)`: the theme exports only data and codly applies it. The blue box (the bar on the left is drawn by a rule of the theme, since codly's border is the same on the four sides), the line numbers in light blue, the file name

on a light blue band, the highlights in the colours of the theme and the syntax colours of `ufac-syntax`. `ufac-theme(codly: codly)` applies it; it is exported for a document that applies codly by itself.

`ufac-syntax` `bytes`

The syntax colours of the code blocks: a tmTheme with the palette of the theme and general scopes (comments in green, strings in red, keywords in blue, numbers in ochre), which suits any language.

8.4 Icons

`icon()`

```
icon(
  name str
  color: auto auto color
  size: 1em length
  baseline: 25% relative
) -> content
```

An icon, already wrapped in a `box` for inline use: a role of the theme (`"warning"`, `"tip"`, `"info"`, `"note"`, `"exercise"`, `"example"`, `"file"`, `"terminal"`, `"contact"`, `"output"`, `"code"`, `"python"`, …), one of the Octicons the theme ships by its 0.2.0 name (`"alert"`, `"light-bulb"`, `"pencil"`, `"beaker"`, …) or any icon of a collection given with `provide-icons`, as `"collection:name"` (`"octicon:rocket-16"`, `"mdi:home"`). An unknown name is a compile error that says how to get the icon.

```
#icon("tip", color: colors.quaternary) A
tip,
#icon("warning") attention and a big
#icon("python", size: 1.6em)
```

A tip, attention and a big

Argument

`name` `str`

Name of the icon.

Argument

`color: auto` `auto` | `color`

Fill color (multicolour logos keep their colours). With `auto` it follows the context like bold text: white inside a pill (a `===` subtitle, an exercise header, the title of an `emph-box()`), the box or title color inside `emph-box()` and `==`, black elsewhere.

Argument

`size: 1em` `length`

Height of the icon; the width follows its proportions (square for the icons of the theme).

Argument

`baseline: 25%` `relative`

Baseline of the box; the default aligns the icon with the text.

```
#emph-box(color: colors.tertiary)[
  #icon("warning") takes the box color.
]
```

takes the box color.

The theme ships only the icons it uses, from Octicons (© GitHub Inc., MIT license), Phosphor (MIT license), Devicon (MIT license) and Simple Icons (CC0); each one has a role in the theme, which is also its name for `#icon`:

warning (octicon:alert-16) · tip (octicon:light-bulb-16) · info (octicon:info-16) · note (octicon:note-16) · exercise (octicon:pencil-16)
example (octicon:beaker-16) · file (octicon:file-code-16) · terminal (octicon:terminal-16) · contact (octicon:mail-16) · output (ph:article-bold)
code (ph:code-bold) · python (devicon:python) · bash (devicon:bash) · matlab (devicon:matlab) · typst (simple-icons:typst)

The Octicons of the theme keep their 0.2.0 names (`"alert"`, `"light-bulb"`, `"info"`, `"note"`, `"pencil"`, `"beaker"`, `"file-code"`, `"terminal"`, `"mail"`). Any other icon comes from a collection in the JSON format of Iconify, given to `provide-icons` and written as `"collection:name"`:

- **The whole collection**: download its JSON from the site of the iconify package (some are several megabytes).
- **Only the chosen icons**: ask the Iconify API for them, such as `https://api.iconify.design/octicon.json?icons=rocket-16,tools-16`, and save the answer as `octicon.json`.

```
#provide-icons(json("octicon.json"))
#icon("octicon:rocket-16") launch
```

The names of a collection are on icones.js.org. An Octicon of 0.2.0 that the theme no longer ships, such as `#icon("rocket")`, is written `#icon("octicon:rocket-16")` with its collection provided.

`theme-icons` `dictionary`

The icon of each role of the theme, as `"collection:name"`: the Octicons of 0.2.0 (size 16) for the callouts, the exercise and example pills, the file name of a code block, the terminal and the contact; Phosphor in the bold weight for the output of a program and for code; the logos of Devicon (Typst from Simple Icons) for the languages. A role is also a valid name for `icon`: `#icon("warning")`.

`provide-icons` `content`

Gives the deck more icons: one or more collections in the JSON format of Iconify (`json("mdi.json")`), whose icons are then written `#icon("mdi:home")`. A collection can be downloaded whole from the site of the iconify package, or only the chosen icons from the Iconify API: `https://api.iconify.design/octicon.json?icons=rocket-16,tools-16`. It is the `provide-icons` of the iconify package, re-exported.

```
#provide-icons(json("octicon.json"))
#icon("octicon:rocket-16")
```

9 Touying features

The theme sits on Touying and leaves its features alone. Animations work on every element of the theme: covered content stays faintly visible under a veil in the page color.

```
== Animations

First this.

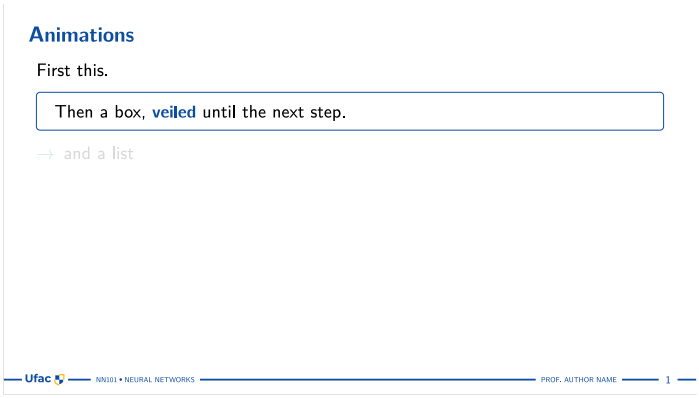
#pause

> Then a box, *veiled* until the
next step.

#pause

-> and a list
```







Keep `#pause` and `#uncover` / `#only` in separate paragraphs: within one paragraph Touying reorders the covered part on the first subslide.

Available without any wrapper from the theme:

Feature	Functions
Animations	`#pause`, `#meanwhile`, `#jump`, `#uncover`, `#only`, `#alternatives`, `#item-by-item`, `#effect`, `#touying-raw`

Feature	Functions
Notes and handouts	`#speaker-note` (it attaches to the previous slide), `#handout-only`, `config-common(handout: true)`
Layout	`#slide(composer: (1fr, 2fr))`, `#cols`, `components.adaptive-columns`, `components.full-width-block`, `utils.fit-to-width`, `utils.fit-to-height`
Overflow	`config-common(breakable: false, clip:, detect-overflow:)`
Reuse	`#touying-recall`, `touying-get-config`, `components.progressive-outline`

Slides of other Touying themes that this one does not have: `outline-slide` (here, a slide with `#outline()`, see Section 2), `focus-slide`, `ending-slide` and `matrix-slide`.

10 Index of definitions

The reference of every function and variable sits in the chapter that introduces it; it is generated from the doc-comments of the package with tidy. Names that start with an underscore are internal and left out.

ufac.typ

answer p. 17
ending-slide() p. 21
example-slide() p. 17
exercise-slide() p. 16
new-section-slide() p. 22
slide() p. 20
solution p. 17
title-slide() p. 6
ufac-theme() p. 4

components.typ

arrows() p. 26
callout() p. 30
cols() p. 34
emph-box() p. 28
eq-box() p. 29
icon() p. 39
outline() p. 13
primary() p. 24
quaternary() p. 25
quote-box() p. 32
secondary() p. 24
tag() p. 31
tertiary() p. 25

icons.typ

provide-icons p. 40
theme-icons p. 40

code.typ

code-languages p. 38
codly-ufac-theme p. 38
ufac-syntax p. 39

info.typ

info() p. 3

constants.typ

colors p. 35
ufac-logo p. 35
ufac-logo-image() p. 36